

Lista de Exercicios

Query Methods e @Query — Spring Data JPA

20 exercicios divididos em **Facil**, **Medio** e **Dificil**. Para cada um, crie o metodo no repositório indicado, rode a aplicacao pelo IntelliJ e acesse `http://localhost:8080` para conferir se ficou verde (PASSOU). O nome exato do metodo que o verificador espera esta anotado no comentario de cada arquivo de repositório — abra o repositório, encontre o exercicio e implemente a partir dali.

FACIL

E1 · Buscar produtos por nome exato

Repositorio: `ProdutoRepository`

Retorne todos os produtos cujo nome seja **exatamente igual** ao informado.

Dica: Comece pelo prefixo `findBy` + o nome da propriedade.

E2 · Produtos com preco acima de um valor

Repositorio: `ProdutoRepository`

Retorne os produtos com preco **maior** que R\$ 300.

Dica: O sufixo `GreaterThan` gera o operador `>`.

E3 · Produtos ativos

Repositorio: `ProdutoRepository`

Retorne apenas os produtos com o campo `ativo = true`.

Dica: Para booleanos existe o sufixo `True` (e `False`).

E4 · Clientes por estado

Repositorio: `ClienteRepository`

Retorne todos os clientes de um determinado estado (UF). Teste com "SP".

Dica: Mesma ideia do E1: o prefixo `findBy` combinado com a propriedade que voce quer filtrar.

E5 · Produtos de uma categoria (propriedade aninhada)

Repositorio: `ProdutoRepository`

Retorne os produtos de uma categoria, filtrando pelo **nome** da categoria.

Dica: Da para filtrar por uma propriedade da entidade **relacionada** (navegue de Produto ate a Categoria).

E6 · Busca por parte do nome (ignorando maiusculas)

Repositorio: `ProdutoRepository`

Retorne produtos cujo nome **contenha** o termo, sem diferenciar maiuscula/minuscula. Teste com "livro".

Dica: Combine `Containing` + `IgnoreCase`.

E7 · Pedidos por status

Repositorio: PedidoRepository

Retorne todos os pedidos com um determinado status. Teste com ENTREGUE.

Dica: O parametro e do tipo enum StatusPedido.

E8 · Contar produtos de uma categoria

Repositorio: ProdutoRepository

Retorne a **quantidade** de produtos de uma categoria (pelo nome dela). Teste com "Eletronicos".

Dica: O prefixo countBy devolve um numero (long).

MEDIO

E9 · Faixa de preco ordenada

Repositorio: ProdutoRepository

Retorne produtos com preco **entre** 100 e 400, ordenados pelo preco (crescente).

Dica: Between usa dois parametros; OrderBy . . . Asc ordena.

E10 · Clientes nascidos antes de uma data

Repositorio: ClienteRepository

Retorne os clientes com data de nascimento anterior a 01/01/1990.

Dica: O sufixo Before compara datas. Parametro do tipo LocalDate.

E11 · Estoque baixo e ativo (dois filtros)

Repositorio: ProdutoRepository

Retorne produtos com estoque **menor** que 10 e que estejam ativos.

Dica: Voce precisa de duas condicoes ligadas por E: uma comparacao de estoque e a de ativo.

E12 · Top 3 produtos mais caros

Repositorio: ProdutoRepository

Retorne os 3 produtos mais caros, do mais caro para o mais barato.

Dica: Use Top3 logo apos o find.

E13 · Faixa de preco com parametros nomeados

Repositorio: ProdutoRepository

Usando @Query (JPQL), retorne produtos com preco entre :min e :max. Teste com 50 e 150.

Dica: Em JPQL existe um operador de **faixa** de valores; lembre de mapear os dois parametros com @Param.

E14 · Cidade OU estado

Repositorio: ClienteRepository

Usando @Query, retorne clientes de uma cidade **ou** de um estado. Teste com "Curitiba" e "RJ".

Dica: No JPQL use OR.

E15 · Pedidos de um cliente ordenados por data

Repositorio: PedidoRepository

Retorne os pedidos de um cliente (por id), do mais recente para o mais antigo.

Dica: Filtre pelo id do cliente (propriedade navegada) e **ordene** pela data, do mais recente para o mais antigo.

DIFICIL

E16 · JOIN: produtos ja vendidos

Repositorio: ProdutoRepository

Retorne (**sem repetir**) todos os produtos que aparecem em ao menos um item de pedido.

Dica: Navegue de ItemPedido para o produto. Nao esqueca do DISTINCT!

E17 · Subconsulta: produtos nunca vendidos

Repositorio: ProdutoRepository

Retorne os produtos que **nunca** apareceram em nenhum item de pedido.

Dica: Use NOT IN com uma subconsulta sobre ItemPedido.

E18 · Agregacao: total gasto por cliente

Repositorio: PedidoRepository

Retorne a **soma** do valorTotal de todos os pedidos de um cliente (por id). Teste com o cliente 1.

Dica: Use uma funcao de **agregacao** para somar os valores, filtrando pelo id do cliente.

E19 · GROUP BY + HAVING

Repositorio: ClienteRepository

Retorne os clientes cujo total gasto (soma dos pedidos) seja **maior** que R\$ 1000.

Dica: Agrupe por cliente e filtre os **grupos** pela soma dos pedidos (pense em GROUP BY + HAVING).

E20 · Query nativa: acima da media da categoria

Repositorio: ProdutoRepository

Retorne os produtos cujo preco seja **maior** que a media de preco da sua propria categoria.

Dica: Use nativeQuery = true e uma subconsulta correlacionada com AVG().
